

RL Homework 3 problem 2

110612025 魏于翔

March 2026

1 (a)

1.1

We maintain the output of the actor network produces mean and the logarithm of the standard deviation is because what action output is a stochastic action and we need to build a probability distribution to sample that action, so mean and the logarithm of the standard deviation is needed. And the reason why we output $\log \sigma$ instead of σ is to ensure the stability of the standard deviation.

1.2

In reparameterization trick, we have to peel the parameter θ from expectation value, by fixing a standard gaussian distribution $\epsilon \sim N(0, I)$ to sample the noise, and use the reparameterization formula:

$$a = \mu_{\theta}(s) + \sigma_{\theta}(s) \odot \epsilon$$

In line 68, $z = \text{dist.rsample}()$ is to implement the above formula, making z into a differentiable function containing μ , σ and ϵ . Now, the gradient can pass through z , μ , σ to the parameter θ , if we use sample instead of `rsample`, the gradient will stop at that point.

1.3

The reason why it is needed is that we use an unbounded Gaussian as the action distribution, However, in the practice, the actions needs to be bounded to a finite interval. To that end, we apply an invertible squashing function ($\tanh$) to the Gaussian samples, and employ the change of variables formula to

compute the likelihood of the bounded actions. In the other words, let $u \in R^D$ be a random variable and $\mu(u|s)$ the corresponding density with infinite support. Then $a = \tanh(u)$, where $\tanh$ is applied elementwise, is a variable with support in $(-1, 1)$ with density given by

$$\pi(a|s) = \mu(u|s) \left| \det \left(\frac{da}{du} \right) \right|^{-1}$$

Since the Jacobian $\frac{da}{du} = \text{diag}(1 - \tanh^2(u))$ is diagonal, the log-likelihood has a simple form

$$\log \pi(a|s) = \log \mu(u|s) - \sum_{i=1}^D \log(1 - \tanh^2(u_i)) \quad (1)$$

So line 67 $\text{action} = z.\tanh()$ is to restrict the output in the range of $(-1, 1)$, line 68 and line 69 are implementing formula (1) to readjust the probability distribution after non-linear transformation of action output.

2 (b)

2.1

The reason why we maintain 2 Q-networks is to prevent the overestimation bias of Q values, which is a common issue in AC methods where function approximation errors accumulate through bootstrapping. Inspired by TD3 algorithm, SAC also utilizes the Clipped Double-Q trick (CDQ), which takes the minimum of the two Q-networks output when compute the bellman target. This ensure a pessimistic and stable value estimation, preventing the policy from exploiting over-optimistic Q-values.

2.2

The two Q-networks we maintained are Q1 and Q2, and they're loss function respectively are

$$L(\theta_1) = E_{(s,a) \in D} [\frac{1}{2} (Q_{\theta_1}(s, a) - y)^2] \quad (2)$$

$$L(\theta_2) = E_{(s,a) \in D} [\frac{1}{2} (Q_{\theta_2}(s, a) - y)^2] \quad (3)$$

y is the target that we want the Q networks to learn, and y is $y = E_{s'}[r + \gamma V_{\phi}(s'|_{s,a})]$, but we want to use Q function to substitute V function by using $V(s') = E_{a' \in \pi}[Q(s', a') - \alpha \log \pi(a', s')]$, put this back to the target y value, and we can get the loss function

$$y = r + \gamma(1-d)E_{a' \in \pi(\cdot|s)}[\min_{j=1,2} Q_{\theta_j}(s', a') - \alpha \log \pi(a'|s')]$$

Substitute this y into formula (2) and (3), then we can get Q1 and Q2's loss function.

3 (c)

The logic behind the auto-tuning of parameter α is we translate the tuning process into optimal problem. We don't need to directly restrict each time step's policy, instead we want in the whole training process, the average entropy is to reach some Target Entropy $\bar{\mathcal{H}}$, in that condition, our goal is to maximize the reward.

At the end of the section 5, it derive the optimal α in each time step in equation (17)

$$\alpha_t^* = \arg \min_{\alpha_t} E_{\alpha_t \in \pi_t^*} [-\alpha_t \log \pi_t^*(a_t|s_t) - \alpha_T \bar{\mathcal{H}}]$$

We want to turn the equation into the objective function of minimizing α .

$$J(\alpha) = E_{a_t \sim \pi_t} [-\alpha \log \pi_t(a_t|s_t) - \alpha \bar{\mathcal{H}}]$$

In implementation, in case of gradient descent makes α less than 0, we will let neural network to optimize the $\log \alpha$, and use exp function to get α .

4 (d)

4.1 hyperparameters

- lr = 3e-4

- batch-size = 256
- memory-size = 50000
- num-steps = 100000

4.2 Results

```

Eval @ step 50000 | Mean Return = -162.10 ± 70.27 over 20 episodes
50% | 50000/100000 [22:39<1:05:09, 12.79it/s]
Episode 251 (Total step = 50001): Total Reward = -124.95578800629207
50% | 50200/100000 [22:42<10:45, 77.19it/s]
Episode 252 (Total step = 50201): Total Reward = -126.38418990132524
50% | 50393/100000 [22:44<10:37, 77.82it/s]
Episode 253 (Total step = 50401): Total Reward = -129.7069650171345
51% | 50596/100000 [22:48<10:59, 74.88it/s]
Episode 254 (Total step = 50601): Total Reward = -248.22564115806426
51% | 50796/100000 [22:50<10:55, 75.10it/s]
Episode 255 (Total step = 50801): Total Reward = -1.0032155338987645
51% | 50996/100000 [22:53<11:08, 73.33it/s]
Episode 256 (Total step = 51001): Total Reward = -247.6162478046589
51% | 51198/100000 [22:58<11:11, 72.64it/s]

```

Figure 1:

The Mean return reaches -162 in 50000 training steps.



Figure 2: Mean return

Above are the mean return in the training process in 100000 training steps.

5 (e)

5.1 hyperparameters

- lr = 3e-4
- batch-size = 512
- NN-architecture = 256->256

- memory-size = 500000
- num-steps = 1000000

5.2 results

Because of the time issue, i only run 900000 training steps, but the mean return reaches 270 when the training steps reach 500000.

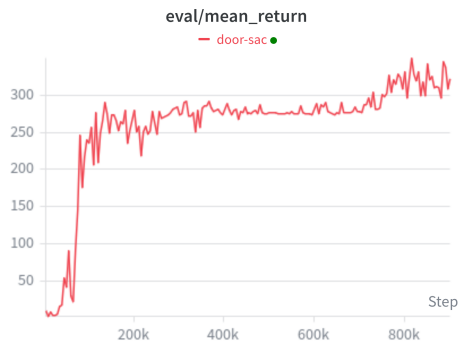


Figure 3: Caption